



INDUSTRIAL DG MONITORING AND ANALYTICS SYSTEM

EXERCISE NO – 15

HARRISH T M	– 24MER016
KAPIL P	– 24MER022
RAMPRASATH D	– 24MER047
SARAVANAKARTHIK S R	– 24MER055
SETHUPATHI M	– 24MER056

15. Industrial DG Monitoring and Analytics System

Project Overview:

A Diesel Generator (DG) is widely used in industries, hospitals, commercial buildings, factories, and other critical facilities as a backup power source. Continuous monitoring of DG operating parameters is important for ensuring reliable operation, reducing downtime, improving maintenance, and detecting abnormal conditions at an early stage.

The Industrial DG Monitoring and Analytics System is an IoT-based monitoring system designed to measure important operating parameters of a diesel generator and display the measured values locally. The system uses an ESP8266 NodeMCU as the main controller and sensors to monitor electrical and environmental parameters.

The proposed system monitors:

- Generator voltage
- Generator current
- Approximate electrical load
- Temperature
- Humidity
- System operating condition

The measured information can be displayed on an OLED display. The ESP8266 can also provide Wi-Fi connectivity so that the monitoring system can be extended to a cloud-based dashboard in the future.

Main Objective:

The main objective of this project is to develop a low-cost embedded monitoring system that can continuously observe important DG parameters and provide useful information about generator operating conditions.

- To monitor DG output voltage.
- To measure the current supplied by the generator.
- To estimate the electrical load.
- To monitor temperature and humidity around the generator.
- To display measured parameters on an OLED display.
- To identify abnormal operating conditions.
- To provide a foundation for IoT-based remote monitoring.
- To improve preventive maintenance and operational safety.

Problem Statement:

Diesel generators are critical backup power sources in industrial environments. However, conventional DG systems often depend on manual inspection and periodic maintenance.

Manual monitoring has several disadvantages:

- Continuous parameters may not be recorded.
- Abnormal voltage or current may go unnoticed.
- Temperature-related problems may not be detected immediately.
- Operators need to physically inspect the generator.
- Historical operating data may not be available.
- Preventive maintenance decisions become difficult without sufficient data.
- Excessive load conditions may not be identified quickly.

Therefore, there is a need for a low-cost monitoring system capable of continuously measuring important DG parameters and presenting the information to the operator.

Proposed Solution:

The proposed solution consists of an ESP8266 NodeMCU, current sensor, voltage sensor, DHT11 temperature/humidity sensor, and OLED display.

The sensors acquire information from the DG. The ESP8266 processes the sensor readings and calculates useful parameters such as electrical power.

The system works as follows:

- The voltage sensor measures generator output voltage.
- The ACS712 measures the current flowing to the load.
- The DHT11 measures temperature and humidity.
- The sensor signals are read by the controller.
- The ESP8266 processes and filters the readings.
- Electrical power is estimated using:
- The measured parameters are displayed on the OLED.
- If a parameter exceeds a predefined limit, the system can indicate a warning.

System Architecture:

The system consists of five major stages:

Stage 1 – Sensing

The sensors collect electrical and environmental data from the generator.

Stage 2 – Signal Acquisition

The sensor outputs are converted into digital values using the ESP8266 ADC or an external ADC such as ADS1115.

Stage 3 – Processing

The ESP8266 processes the sensor readings, applies calibration/scaling, and calculates electrical power.

Stage 4 – Display

The OLED provides real-time information to the operator.

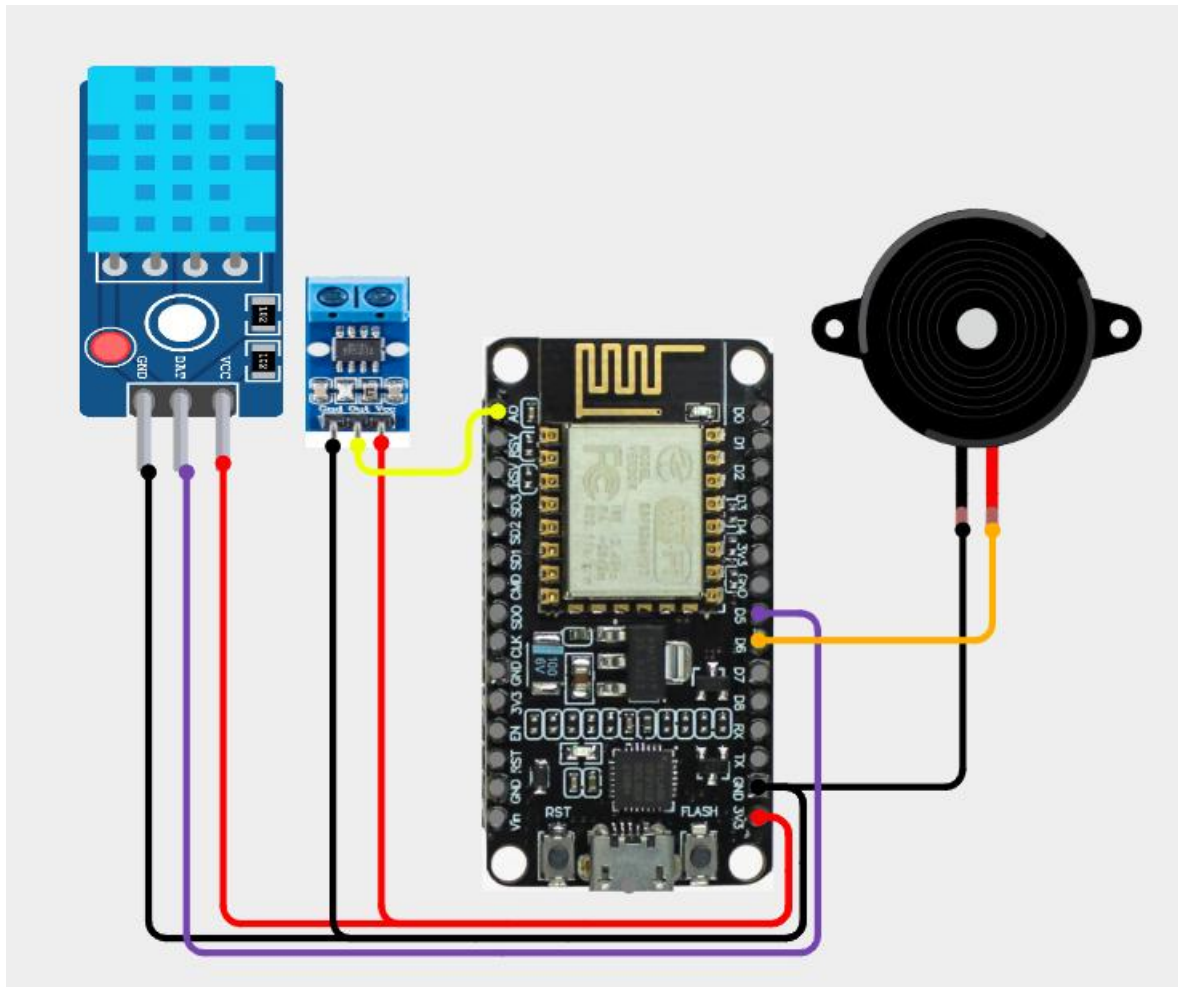
Stage 5 – IoT Communication

The ESP8266 Wi-Fi capability can transmit the information to an online dashboard or server.

Circuit Table:

Component	Pin	NodeMCU / ADS1115 Connection
OLED	VCC	3.3V
OLED	GND	GND
OLED	SDA	D2 / GPIO4
DHT11	SCL	3.3V
DHT11	VCC	D5 / GPIO14
Voltage Sensor	VCC	As specified by sensor

Circuit Design:



Algorithm:

- Start
- Initialize ESP8266, OLED, DHT11 and sensors.
- Read voltage from the voltage sensor.
- Read current from the ACS712.
- Read temperature and humidity from DHT11.
- Calculate electrical power using .
- Compare measured parameters with predefined threshold values.
- If a parameter exceeds its threshold, generate a warning/alert.
- Display the measured parameters on the OLED.
- Transmit or store the data through Wi-Fi.
- Wait for the next sampling interval.
- Repeat the process continuously.

Coding:

```
#include <ESP8266WiFi.h>
#include "AdafruitIO_WiFi.h"
#include <DHT.h>
#define WIFI_SSID "K-APIL"
#define WIFI_PASS "kapil.2333"

#define IO_USERNAME "saravanakarthiks"
#define IO_KEY      "aio_idPg82y91fmNVXL0iJA71zKwH97q"

AdafruitIO_WiFi io(IO_USERNAME, IO_KEY, WIFI_SSID, WIFI_PASS);

AdafruitIO_Feed *currentFeed =
  io.feed("generator-current");

AdafruitIO_Feed *temperatureFeed =
  io.feed("generator-temperature");

AdafruitIO_Feed *humidityFeed =
  io.feed("generator-humidity");

AdafruitIO_Feed *statusFeed =
  io.feed("generator-status");

AdafruitIO_Feed *buzzerFeed =
  io.feed("generator-buzzer");

#define ACS_PIN    A0
#define DHT_PIN    14    // D5 = GPIO14
#define BUZZER_PIN 12    // D6 = GPIO12

#define DHTTYPE DHT11

DHT dht(DHT_PIN, DHTTYPE);

const float MAX_CURRENT = 10.0; // Amperes
const float MAX_TEMP    = 50.0; // Celsius
const float MAX_HUMIDITY = 80.0; // Percentage
```

```
// ACS712-5A = 0.185
// ACS712-20A = 0.100
// ACS712-30A = 0.066

const float ACS_SENSITIVITY = 0.100;

// Zero-current voltage
const float ACS_ZERO_VOLTAGE = 2.50;

// ESP8266 ADC voltage
const float ADC_VOLTAGE = 3.3;

float current = 0;
float temperature = 0;
float humidity = 0;

String generatorStatus;

unsigned long previousMillis = 0;

const unsigned long interval = 5000;

void setup()
{
  Serial.begin(115200);

  delay(1000);

  Serial.println();
  Serial.println("=====");
  Serial.println(" INDUSTRIAL DIESEL GENERATOR MONITORING");
  Serial.println("=====");

  pinMode(BUZZER_PIN, OUTPUT);

  digitalWrite(BUZZER_PIN, LOW);

  dht.begin();

  Serial.println("DHT11 Started");
  Serial.println("ACS712 Started"); Serial.println();
  Serial.println("Connecting to Adafruit IO...");
```

```

io.connect();

while (io.status() < AIO_CONNECTED)
{
    Serial.print(".");
    delay(500);
}

Serial.println();
Serial.println("Adafruit IO Connected!");

Serial.println("-----");

Serial.print("Current Limit   : ");
Serial.print(MAX_CURRENT);
Serial.println(" A");

Serial.print("Temperature Limit : ");
Serial.print(MAX_TEMP);
Serial.println(" C");

Serial.print("Humidity Limit   : ");
Serial.print(MAX_HUMIDITY);
Serial.println(" %");

Serial.println("-----");
}

// READ ACS712 CURRENT

float readCurrent()
{
    const int samples = 100;

    float voltageSum = 0;

    for (int i = 0; i < samples; i++)
    {
        int adcValue = analogRead(ACS_PIN);

        float sensorVoltage =
            (adcValue / 1023.0) * ADC_VOLTAGE;

        voltageSum += sensorVoltage;
    }
}

```

```

    delay(2);
}

float averageVoltage =
    voltageSum / samples;

float currentValue =
    (averageVoltage - ACS_ZERO_VOLTAGE)
    / ACS_SENSITIVITY;

// Convert negative to positive
if (currentValue < 0)
{
    currentValue = -currentValue;
}

// Remove small noise
if (currentValue < 0.15)
{
    currentValue = 0;
}

return currentValue;
}

```

```

void readSensors()
{
    // DHT11
    temperature = dht.readTemperature();
    humidity = dht.readHumidity();

    // ACS712
    current = readCurrent();
}

```

```

void checkStatus()
{
    bool currentHigh =
        current > MAX_CURRENT;

    bool temperatureHigh =

```

```
    temperature > MAX_TEMP;

bool humidityHigh =
    humidity > MAX_HUMIDITY;

if (currentHigh &&
    temperatureHigh &&
    humidityHigh)
{
    generatorStatus = "CRITICAL";

    digitalWrite(BUZZER_PIN, HIGH);
}

else if (currentHigh &&
    temperatureHigh)
{
    generatorStatus = "OVERLOAD + HIGH TEMP";

    digitalWrite(BUZZER_PIN, HIGH);
}

else if (currentHigh)
{
    generatorStatus = "OVERLOAD";

    digitalWrite(BUZZER_PIN, HIGH);
}

else if (temperatureHigh)
{
    generatorStatus = "HIGH TEMPERATURE";

    digitalWrite(BUZZER_PIN, HIGH);
}

else if (humidityHigh)
{
    generatorStatus = "HIGH HUMIDITY";
```

```

        digitalWrite(BUZZER_PIN, HIGH);
    }

    else
    {
        generatorStatus = "NORMAL";

        digitalWrite(BUZZER_PIN, LOW);
    }
}

void sendToAdafruit()
{
    // Current
    currentFeed->save(current);

    // Temperature
    temperatureFeed->save(temperature);

    // Humidity
    humidityFeed->save(humidity);

    // Status
    statusFeed->save(generatorStatus.c_str());

    // Buzzer
    if (generatorStatus == "NORMAL")
    {
        buzzerFeed->save("OFF");
    }
    else
    {
        buzzerFeed->save("ON");
    }

    Serial.println();
    Serial.println("Data sent to Adafruit IO");
}

void printSerialData()
{
    Serial.println();
    Serial.println("=====");
    Serial.println("    DIESEL GENERATOR STATUS");
}

```

```

Serial.println("=====");

Serial.print("Current      : ");
Serial.print(current, 2);
Serial.println(" A");

Serial.print("Temperature   : ");
Serial.print(temperature, 1);
Serial.println(" C");

Serial.print("Humidity      : ");
Serial.print(humidity, 1);
Serial.println(" %");

Serial.println("-----");

Serial.print("Current Limit   : ");
Serial.print(MAX_CURRENT, 2);
Serial.println(" A");

Serial.print("Temperature Limit: ");
Serial.print(MAX_TEMP, 1);
Serial.println(" C");

Serial.print("Humidity Limit  : ");
Serial.print(MAX_HUMIDITY, 1);
Serial.println(" %");

Serial.println("-----");

Serial.print("Generator Status : ");
Serial.println(generatorStatus);

Serial.print("Buzzer        : ");

if (generatorStatus == "NORMAL")
{
    Serial.println("OFF");
}
else
{
    Serial.println("ON");
}

Serial.println("=====");
}

```

```
void loop()
{
  // Keep Adafruit IO connection alive
  io.run();

  unsigned long currentMillis = millis();

  if (currentMillis - previousMillis >= interval)
  {
    previousMillis = currentMillis;

    readSensors();

    if (isnan(temperature) ||
        isnan(humidity))
    {
      Serial.println("ERROR: DHT11 reading failed!");

      digitalWrite(BUZZER_PIN, HIGH);

      delay(300);

      digitalWrite(BUZZER_PIN, LOW);

      return;
    }

    checkStatus();

    printSerialData();

    sendToAdafruit();
  }
}
```

System Working:

Step 1 – Power ON

When the system is powered, the ESP8266 initializes all connected sensors and the OLED display.

Step 2 – Sensor Measurement

The voltage sensor measures the scaled electrical voltage signal.

The ACS712 measures current flowing through the monitored circuit.

The DHT11 measures temperature and humidity.

Step 3 – Data Processing

The ESP8266 processes the sensor outputs and converts them into useful values.

For example:

Voltage = 220 V

Current = 4 A

Power = 220×4

Power = 880 W

For an actual AC DG system, the calculation should use properly measured RMS values and power factor.

Step 4 – Display

The OLED continuously displays:

- Voltage
- Current
- Power
- Temperature
- Humidity
- System status

Step 5 – Condition Monitoring

The controller compares measured parameters against configured limits.

For example:

Normal Voltage → NORMAL

High Current → OVERLOAD WARNING

High Temperature → HIGH TEMPERATURE WARNING

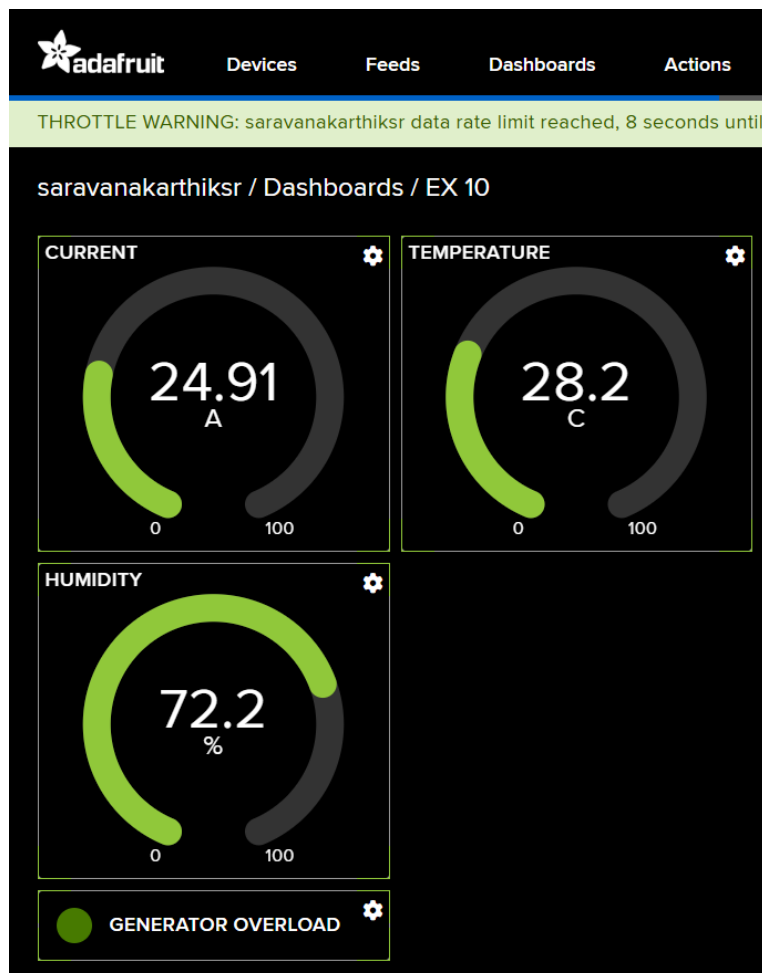
Low Voltage → LOW VOLTAGE WARNING

Step 6 – IoT Monitoring

The ESP8266 can use Wi-Fi to transmit the readings to an IoT platform or server.

This enables:

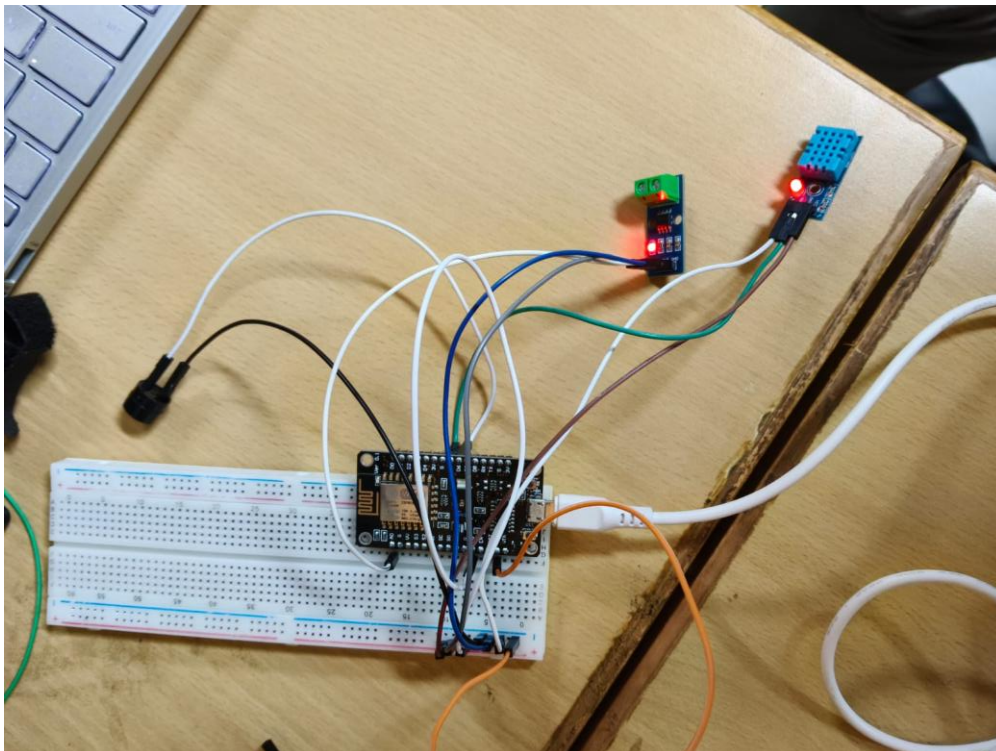
- Remote monitoring
- Data logging
- Graph generation
- Historical analysis
- Notifications/alerts



Bill of Materials:

15. Industrial DG Monitoring and Analytics System	
Component	Quantity
ESP8266 NodeMCU	1
ACS712 Current Sensor	1
Voltage Sensor Module	1
DHT11 Sensor	1
OLED Display	1
Breadboard	1
Jumper Wires	1 Set

Prototype Implementation:



Results & Performance Analysis:

1. Real-time monitoring

The system continuously acquires sensor data and updates the display.

2. Parameter monitoring

The prototype monitors:

- Voltage
- Current
- Power
- Temperature
- Humidity

3. Abnormal-condition detection

Threshold-based logic can identify:

- Overcurrent
- Undervoltage
- Overvoltage
- High temperature

4. IoT capability

The ESP8266 provides Wi-Fi connectivity, allowing the system to be expanded into a remote monitoring system.

5. Low-cost implementation

The proposed prototype uses inexpensive and commonly available electronic components.